

Laboratorio 1 - Clases, datos y expresiones y relaciones entre clases

Objetivo

- Diferenciar entre definición y creación de objetos.
- Correcto manejo de clases y sus relaciones.
- Modelar su programa en UML.
- Aplicar de forma correcta los modificadores de visibilidad para no violar el encapsulamiento.
- Creación de un programa principal que resuelva el problema.
- Identifica los componentes del problema y la solución siguiendo el paradigma de orientación a objetos.

Introducción

En la Liga Elemental, dos entrenadores se preparan para un combate por turnos donde la estrategia importa tanto como la suerte. Cada uno ha entrenado a cuatro Pokémon —de tipo Fuego, Agua, Planta o Eléctrico— y les ha enseñado una habilidad especial que solo se activa en el momento justo. Durante el combate los roles se alternan: en cada ronda uno de los entrenadores toma la ofensiva mientras el otro se prepara para resistir el golpe, y en la ronda siguiente los papeles se invierten. Cuatro rondas después, quien haya ganado más enfrentamientos se corona campeón.

Antes del combate

1. Cada entrenador escoge 4 Pokémon distintos al inicio (pueden crearse directamente en el código, no es necesario pedirlos por teclado).
2. Cada Pokémon tiene:
 - Nombre
 - Tipo (Fuego, Agua, Planta, Eléctrico)
 - Ataque
 - Defensa
 - Habilidad Especial, con:
 - Nombre
 - Probabilidad de activación (ej. 30% → se genera un número aleatorio entre 0 y 100; si cae en el rango 0-30 inclusive, la habilidad se activa)

Toda habilidad especial funciona igual: es un potenciador de hasta 25% que se suma al ataque del Pokémon si está atacando esa ronda, o a su defensa si está defendiendo. El detalle de cómo y cuándo se calcula está en la siguiente sección.

Turnos: quién ataca y quién defiende

El combate consta de 4 rondas y los roles de atacante y defensor se alternan en cada una, siguiendo siempre este orden fijo:

- Ronda 1: Jugador 1 ataca — Jugador 2 defiende
- Ronda 2: Jugador 2 ataca — Jugador 1 defiende
- Ronda 3: Jugador 1 ataca — Jugador 2 defiende
- Ronda 4: Jugador 2 ataca — Jugador 1 defiende

Cada entrenador ataca exactamente 2 veces y defiende exactamente 2 veces durante el combate. El programa debe llevar el control de quién tiene el rol de atacante en cada ronda (por ejemplo, alternando una bandera en cada iteración del ciclo de rondas).

Desarrollo de cada ronda

1. Selección de Pokémon (aleatoria): tanto el atacante como el defensor seleccionan al azar uno de sus Pokémon que no haya peleado todavía. Con 4 Pokémon y 4 rondas, cada Pokémon participa exactamente una vez.
2. Intento de habilidad (aleatorio): los dos Pokémon en juego —el atacante y el defensor— hacen cada uno su propio sorteo para saber si su habilidad se activa (número aleatorio entre 0 y 100 comparado contra su probabilidad de activación).
3. Magnitud del potenciador (aleatoria): por cada habilidad que sí se activó, se genera un segundo número aleatorio, esta vez entre 0 y 25. Ese número es el porcentaje de potenciador que otorga esa habilidad en esta ronda.
4. Aplicación del potenciador: el porcentaje obtenido se suma como bono directamente sobre la estadística que corresponde al rol del Pokémon en esta ronda —nunca sobre la otra—:
 - Si el Pokémon con la habilidad activa es el atacante → su ataque de esta ronda aumenta ese porcentaje.
 - Si el Pokémon con la habilidad activa es el defensor → su defensa de esta ronda aumenta ese porcentaje.

Por ejemplo, si el atacante tiene 40 de ataque y su habilidad se activa con un potenciador de 20%, su ataque efectivo en esta ronda es $40 + (40 \times 0.20) = 48$.

5. Cálculo del resultado de la ronda:

ataqueTotal = ataque efectivo del atacante + bono de efectividad de tipo – defensa efectiva del defensor

- **ataqueTotal > 0** → gana la ronda el atacante (su ataque superó la resistencia del rival).
- **ataqueTotal < 0** → gana la ronda el defensor (logró resistir el ataque).
- **ataqueTotal = 0** → empate en la ronda.

Gana la batalla quien gane más de las 4 rondas; si terminan 2 a 2, la batalla se declara empatada.

Efecto de Tipo

La bonificación de tipo depende de qué tipo ataca a qué tipo, según la siguiente matriz (fila = tipo que ataca, columna = tipo que defiende):

Ataca ↓ / Defiende →	Fuego	Agua	Planta	Eléctrico
Fuego	0	-10	+20	0
Agua	+20	0	-10	0
Planta	-10	+20	0	0
Eléctrico	0	+20	0	0

- **Efectivo (+20):** el Pokémon que ataca tiene ventaja de tipo sobre el que defiende (ej. Fuego ataca Planta).
- **Débil (-10):** el Pokémon que ataca tiene desventaja de tipo frente al que defiende (ej. Planta ataca Fuego).
- **Neutral (0):** no hay ventaja ni desventaja de tipo, incluyendo cuando un Pokémon ataca a otro de su mismo tipo, y todos los enfrentamientos de Eléctrico no señalados arriba como efectivos (ej. Eléctrico ataca Fuego, Eléctrico ataca Planta).

Habilidad Especial

Ejemplos de nombres y probabilidad de activación (la magnitud del potenciador, entre 0% y 25%, siempre se calcula al momento de activarse, como se explicó arriba):

- “Llama Final” → 30% de probabilidad de activación
- “Escudo Natural” → 40% de probabilidad de activación
- “Impacto Relámpago” → 20% de probabilidad de activación

No existen habilidades con efectos distintos entre sí: todas son el mismo tipo de potenciador (hasta 25%) y lo único que las distingue es su nombre y su probabilidad de activación.

Análisis

Cree una tabla en la que se incluya toda la información solicitada. Puede hacer una tabla para propiedades y otra para métodos:

1. ¿Qué propiedades y métodos tendrá cada clase?
2. ¿Qué tipo de datos deben tener las propiedades y métodos de cada clase?
3. ¿Cuáles deben ser los modificadores de visibilidad de los miembros en cada clase?
4. ¿Qué parámetros serán requeridos por los métodos en sus clases?
5. ¿Qué propósito tendrá dentro de la clase?

Material a entregar en Canvas

- Archivo PDF con el análisis y diseño.
 - (No incluya URL del diagrama sino en vez adjúntelo en el pdf o como un archivo externo). Use la plantilla adjunta.
- Url al repositorio público en GitHub conteniendo todos los archivos .java.

Evaluación

Análisis (30 puntos)

- (15 puntos) Tabla de atributos de todas las clases.
- (15 puntos) Tabla de métodos de todas las clases.

Diseño (30 puntos)

- (15 puntos) Están representados cada uno de los componentes de la clase de manera correcta, clasificados por color según su rol en MVC, y coinciden con lo
- Las relaciones entre las clases están correctas, utilizando agregación o composición según corresponda (no se acepta asociación genérica).

Programa (40 puntos)

- (20 puntos) La cantidad de clases identificadas es suficiente para resolver el problema planteado.
- (20 puntos) El problema se resolvió empleando programación orientada a objetos. Se almacenó la información necesaria creando y llenando objetos de las clases identificadas.

Notas

- Para obtener una nota distinta de 0 en la parte del programa debe existir una congruencia entre el diseño y el programa. Una variación considerable (criterio subjetivo) resultará en la anulación de la parte del programa.
- Para optar a que se califique la parte del programa es necesario que el merge request lo hagan a su branch dentro del repo del curso; commits a branches equivocadas resulta en que se rechace el pull request y no se califique el código